ATIF-Gemma: An Open Empirical Diagnostic Corpus and Failure Taxonomy for Small Open-Weight Developer Agents

Karandeep Singh Sodhi

Independent Researcher

karandeep.sodhi2504a@gmail.com

October 27, 2026

Track: Google — The Gemma 4 Developer Agent Competition (Paper Track: Best New Resource)

Author Note & Resource Manifest

Karandeep Singh Sodhi is an independent researcher.

Open Research Resource: All 64 full-fidelity execution traces formatted in Agent Trajectory Interchange Format (ATIF v1.7), ground-truth reference specifications, metadata index, and diagnostic validation scripts are openly released on Kaggle and GitHub under the Creative Commons Attribution 4.0 International (CC BY 4.0) license.

Kaggle Dataset: https://www.kaggle.com/datasets/karandeepsinghsodhi/atif-gemma-swe-agent-traces
GitHub Repository: https://github.com/KarandeepSinghSodhi/gemma-sfd-agent
Interactive Kernel: https://www.kaggle.com/code/karandeepsinghsodhi/atif-gemma-diagnostic-trace-exploration

Disclosures: The author declares no competing financial or non-financial interests.

Correspondence: Correspondence concerning this article should be addressed to Karandeep Singh Sodhi, Email: karandeep.sodhi2504a@gmail.com.

Abstract

Current software engineering (SWE) agent benchmarks report scalar pass/fail resolution rates, obscuring the multi-stage dynamics of agent failure. For small, quantized open-weight models such as Gemma 4 E4B, understanding why agents plateau at low resolution rates (median 5.0% across 197 teams on the official Small Model Prize leaderboard) is hindered by a complete absence of publicly available, fine-grained execution trace data. We introduce **ATIF-Gemma**, an open empirical diagnostic corpus of 64 full-fidelity multi-turn execution traces comprising 847 LLM calls, 660 tool invocations, and 6,378,543 logged tokens generated across 16 paired SWE-bench tasks on 4x NVIDIA L4 GPUs inside the official Google ADK and `swegemma` execution harness.

Formatted in the Agent Trajectory Interchange Format (ATIF v1.7), ATIF-Gemma cross-references every step against ground-truth repository patches and held-out test patches. We establish an empirical 4-way Phase 2 exit code failure taxonomy: while 93.8% of runs (60/64) successfully synthesize a non-empty patch, 31.2% (20/64) exit with collection/import errors (`exit_code = 2`) due to sandbox dependency absences, global venv shadowing, and detached syntax blocks; 56.2% (36/64) exit with test assertion failures (`exit_code = 1`) due to partial localization or imprecise edits; 6.2% (4/64) produce no patch due to budget or timeout exhaustion (`exit_code = -1`); and 6.2% (4/64) achieve full resolution (`exit_code = 0`). We release the raw trace corpus, structured metadata tables, ground-truth reference mappings, and an open diagnostic Python tooling suite to provide the community with an empirical benchmark for small-model agent research.

Keywords: agent trajectory corpus, SWE-bench, Gemma 4, failure taxonomy, ATIF v1.7, open-weight agents, error diagnostics

1. Introduction & The Diagnostic Vacuum in SWE Benchmarking

Autonomous software engineering agents powered by large language models have achieved notable progress on repository-level repair benchmarks such as SWE-bench [1]. However, benchmarking in this domain remains almost universally **outcome-centric**: evaluations report a single scalar accuracy metric (the proportion of tasks resolved), discarding the rich, high-dimensional trajectory of decisions, tool calls, token expansions, and environmental interactions that preceded the outcome.

For frontier models (e.g. Claude 3.5 Sonnet, GPT-4o) and large open-weight models (Gemma 4 31B), high resolution rates (22.0%–24.0% on the Google Gemma 4 challenge) allow researchers to treat the agentic loop as an effective black box. In contrast, for compact, 4-bit quantized open-weight models suitable for local developer workstations (such as `gemma-4-e4b-it-qat-w4a16-ct`), scalar benchmarking reveals an acute capability cliff without explaining its origins. Across all $N = 197$ participating teams on the official Small Model Prize leaderboard (ryanholbrook/gemma-4-developer-agent-small-model-leaderboard), public resolution scores range from 0.0% to 10.0%, with a median of exactly 5.0%, a mean of 4.23%, and 183 of 197 teams (92.9%) scoring $\leq 6.0\%$.

When an agent fails, developers currently cannot discern whether failure stemmed from:

1. **Upstream Localization Failure**: Did the agent search the wrong directories or read irrelevant modules?
2. **Context Saturation & Syntax Corruption**: Did prompt bloat degrade the model's instruction-following fidelity, causing malformed tool arguments or detached code blocks?
3. **Environmental & Sandbox Artifacts**: Did the patch fail held-out tests due to missing third-party packages in the offline sandbox, or global virtual environment packages shadowing local repository source files?
4. **Tool Exhaustion**: Did the agent exhaust its global turn, time, or tool limits before applying an edit?

To bridge this diagnostic gap, we introduce **ATIF-Gemma**, an open, standardized, full-fidelity diagnostic trajectory corpus and software toolkit for small open-weight developer agents. We provide 64 complete

execution traces totaling 6,378,543 tokens, 847 LLM turns, and 660 tool invocations collected on real 4x NVIDIA L4 GPUs, structured under the Agent Trajectory Interchange Format (ATIF v1.7).

2. Corpus Architecture & ATIF v1.7 Schema Specifications

Corpus Partition	Traces	Tasks	LLM Calls	Tool Calls	Total Tokens
Cohort 1 (Canonical Paired)	24	8	306	242	2,333,702
Cohort 2 (Stratified Random)	24	8	304	263	2,642,882
Cohort 1 Rerun (Reproducibility)	16	8	237	155	1,401,959
Total ATIF-Gemma Corpus	**64**	**16**	**847**	**660**	**6,378,543**

Table 1: Summary composition of the ATIF-Gemma diagnostic trace corpus across paired cohorts on 4x NVIDIA L4 GPUs.

The corpus covers 16 paired SWE-bench instances spanning four major Python repositories: `fastapi` (27 runs), `rich` (19 runs), `requests` (13 runs), and `httpx` (5 runs). All experiments were served using vLLM 0.19.1 under tensor parallel 4 (`tp=4`, 96 GB aggregate VRAM) hosting `gemma-4-e4b-it-qat-w4a16-ct` with Google ADK 1.36.1 and `swegemma`.

Each trace conforms strictly to the **ATIF v1.7 JSON Schema**:

- **Metadata Block**: Records `instance_id`, `repo`, `base_commit`, `agent_config_name`, vLLM sampling parameters (`temperature=0.0`, `seed=42`, `max_output_tokens=8192`, `thinking_budget=1536`), and hardware fingerprints.
- **Step Sequence**: Sequential array of interaction turns containing:
  - `metrics`: Exact per-call `prompt_tokens`, `completion_tokens`, and `total_tokens` reported directly by the vLLM engine.
  - `tool_calls`: Function name, input argument JSON dictionary, and stage author tag (`extra.author`).
  - `observation`: Raw tool execution stdout/stderr response, JSON-parsed error status, and error classification types (`FileEditError`, `CommandError`, `FileReadError`, `BudgetExceeded`).
  - `thought`: Full model reasoning trace emitted within the thinking budget.
- **Final Metrics**: Captures cumulative token counts, total duration in seconds, agent patch string, and Phase 2 test execution exit codes.

3. The 4-Way Phase 2 Exit Code Failure Taxonomy

In SWE-bench, patch verification is executed in an isolated verification sandbox using `pytest`. In ATIF-Gemma, cross-referencing agent traces against Phase 2 test runner outputs reveals that treating unresolved runs as a monolithic failure category conceals fundamentally different failure mechanics. We define an empirical 4-way taxonomy:

Category	Exit Code	Count (%)	Empirical Mechanism & Observable Symptoms
Resolved	0	4 (6.2%)	Patch passes all Phase 2 held-out unit tests with zero regressions.
Assertion Failure	1	36 (56.2%)	Patch syntactically valid and executes under pytest, but fails assertions due to wrong logic or incomplete multi-file edits.
Collection Error	2	20 (31.2%)	Pytest crashes during test collection prior to assertion evaluation (ModuleNotFoundError, SyntaxError, venv shadowing).
Budget / Timeout	−1	4 (6.2%)	Agent exhausted 15 tool calls or 2.5 min wall time; 0 patch characters submitted.

Table 2: Empirical 4-way Phase 2 exit code distribution across all 64 ATIF-Gemma execution traces.

3.1 Analysis of Collection / Import Errors (Exit Code 2: 31.2%)

A critical finding enabled by ATIF-Gemma is that **nearly one-third of all agent runs (20/64) failed before unit tests could even execute**. Our trace cross-referencing identifies three distinct sub-mechanisms:

1. **Missing Sandbox Dependencies**: In tasks such as `fastapi_14361` and `fastapi_14459`, the held-out `test_patch` imports `inline_snapshot`. Because `inline_snapshot` was not packaged into the offline wheelhouse, pytest terminates with `ModuleNotFoundError` during test collection across all evaluated configurations.
2. **Environment Shadowing**: On `httpx_3672`, the offline sandbox environment resolved `import httpx` to `/usr/local/lib/python3.12/dist-packages/httpx` (pre-installed system package) rather than the local repository source at `/workspace/src/httpx`, rendering source edits invisible during test collection.
3. **Syntactic Corruption**: In `fastapi_14099`, `PatchAgent` inserted an unindented loop that left a detached `else:` block at line 259, raising `SyntaxError: invalid syntax` during collection.

3.2 Ground-Truth Localization Drift vs. Reference Patches

By cross-referencing all 16 unique evaluated tasks against ground-truth reference patches in `tasks.jsonl`, ATIF-Gemma categorizes localization accuracy into three distinct tiers:

- **Exact Single-File Localization (4/16 tasks = 25.0%)**: `rich_4077` (resolved), `fastapi_15588`, `fastapi_15589`, and `rich_3043`.
- **Partial Multi-File Localization (6/16 tasks = 37.5%)**: Multi-file bugs where the agent localized only a single site (e.g. `fastapi_14099` [1/3 files], `fastapi_14459` [1/3 files], `requests_7502` [1/2 sites], `requests_7505` [1/2 files], `rich_4070` [1/9 files], and `httpx_3672` [1/7 files]).
- **Wrong File / Function Localization (6/16 tasks = 37.5%)**: Complete localization divergence (`fastapi_14361`, `fastapi_15661`, `fastapi_9753`, `requests_7427`, `rich_3944`, `rich_4076`).

4. Quantitative Audit Across Agent Architectures

ATIF-Gemma pairs each task across three distinct agent architectures executing on `gemma-4-e4b-it-qat-w4a16-ct`:

Configuration	Runs	Resolved	Mean Peak P_t	Mean Cum. Tokens	Mean Duration
Official Starter (flat_starter)	16	1/16 (6.2%)	13,029.3	103,350.1	97.3s
Single-Agent (flat_controlled)	16	0/16 (0.0%)	13,550.8	119,664.4	92.2s
State-Filtered Deliberation (sfd)	16	1/16 (6.2%)	11,291.9	88,909.5	89.4s
Rerun: flat_controlled	8	1/8 (12.5%)	13,388.5	119,776.4	86.9s
Rerun: sfd	8	1/8 (12.5%)	11,048.8	86,816.6	87.6s

Table 3: Architectural comparison of token consumption, peak prompt lengths, and execution speed across ATIF-Gemma partitions.

4.1 Sub-Agent Bypassing in the Starter Kit

ATIF-Gemma provides the first empirical trace evidence of how small models handle discretionary sub-agents. The official starter kit wraps `code_analyzer` as an `AgentTool`. In ATIF-Gemma's 16 `flat_starter` traces:

- Out of 149 non-submit tool decisions, the root agent delegated to `code_analyzer` only **4 times (2.7%)**.
- The model completely **bypassed `code_analyzer` on 13 of 16 tasks (81.2%)**, opting instead to read large files directly into its primary context.

4.2 Cross-Stage Prompt Resets

In `sfd` traces, configuring `include_contents: "none"` on downstream stages resets entry prompts from a mean TriageAgent peak of **10,922.1 tokens** down to **1,500.9 tokens** at PatchAgent entry (an **−86.3% cross-boundary reset**), cutting cumulative token burn by **25.7%** relative to single-agent baselines ($p = 0.0213$).

5. The ATIF-Gemma Diagnostic Toolkit

Along with the dataset, we release a standalone Python package (`atif_tools`) providing:

1. `audit_traces(trace_dir)`: Automatically parses any ATIF v1.7 trajectory directory, extracting token trajectories, tool error frequencies, and exit-code failure buckets.
2. `check_consistency.py`: Automated zero-drift validation script asserting exact equality between paper tables and raw trace files.
3. `reproduce_eval.py`: Single-command runner for re-executing trajectories under vLLM with frozen random seeds.

6. Limitations & Threats to Validity

ATIF-Gemma records 64 paired runs on 16 tasks ($n = 16$). While statistically balanced and stratified across repositories, evaluating all 129 SWE-bench instances would require 500 GPU hours. Second, tests were conducted specifically on 4-bit quantized Gemma 4 E4B; while representative of small open-weight developer agents, frontier models may display different failure distributions.

7. Conclusion & Resource Availability

ATIF-Gemma establishes an open, reproducible, full-fidelity diagnostic foundation for the small-model coding agent community. By moving beyond scalar pass/fail metrics to an open 64-trace ATIF v1.7 corpus and a 4-

way Phase 2 exit code taxonomy, this resource enables researchers to target the real bottlenecks of small developer agents: sandbox collection errors, context bloat, and localization drift.

Dataset URL: https://www.kaggle.com/datasets/karandeepsinghsodhi/atif-gemma-swe-agent-traces
Code Repository: https://github.com/KarandeepSinghSodhi/gemma-sfd-agent

References

[1] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *International Conference on Learning Representations (ICLR)*, 2024.